PRODUCT INVENTORY MANAGEMENT SYSTEM

Backend API Project Documentation

Team ID : SWTID-2026-2428

Team Leader : LOGESHWARI K T

Team member : ALJASIRA R

Team member : KAVIYA N

Team member : MONIKA S

Team member : PREETHA S

Course: Backend Development

Technology: Node.js, Express.js, MongoDB

Table of Contents

1. Project Overview

The Product Inventory Management System is a backend application designed to manage products using RESTful APIs. The system is developed using Node.js, Express.js and MongoDB and uses JWT authentication to secure the APIs.

2. Objectives

Provide secure authentication using JWT tokens

Implement CRUD operations for product management

Allow users to manage their own products

Build scalable backend architecture using Node.js

Enable easy API testing using Postman

3. System Architecture

The system follows a three-tier architecture consisting of Client, Backend API, and Database.

```
Client (Postman)
       |
       v
Node.js + Express Server
       |
Controllers → Routes → Middleware
       |
       v
MongoDB Database
```

4. Technology Stack

Technology	Purpose
Node.js	Backend runtime environment

Express.js	Web framework for API development
MongoDB	Database for storing users and products
Mongoose	ODM for MongoDB
JWT	Authentication mechanism
Postman	API testing tool

5. Coding Implementation

Basic Server Setup Example:

```
const express = require('express');
const mongoose = require('mongoose');
const app = express();

app.use(express.json());

app.listen(5000, () => {
  console.log("Server running on port 5000");
});
```

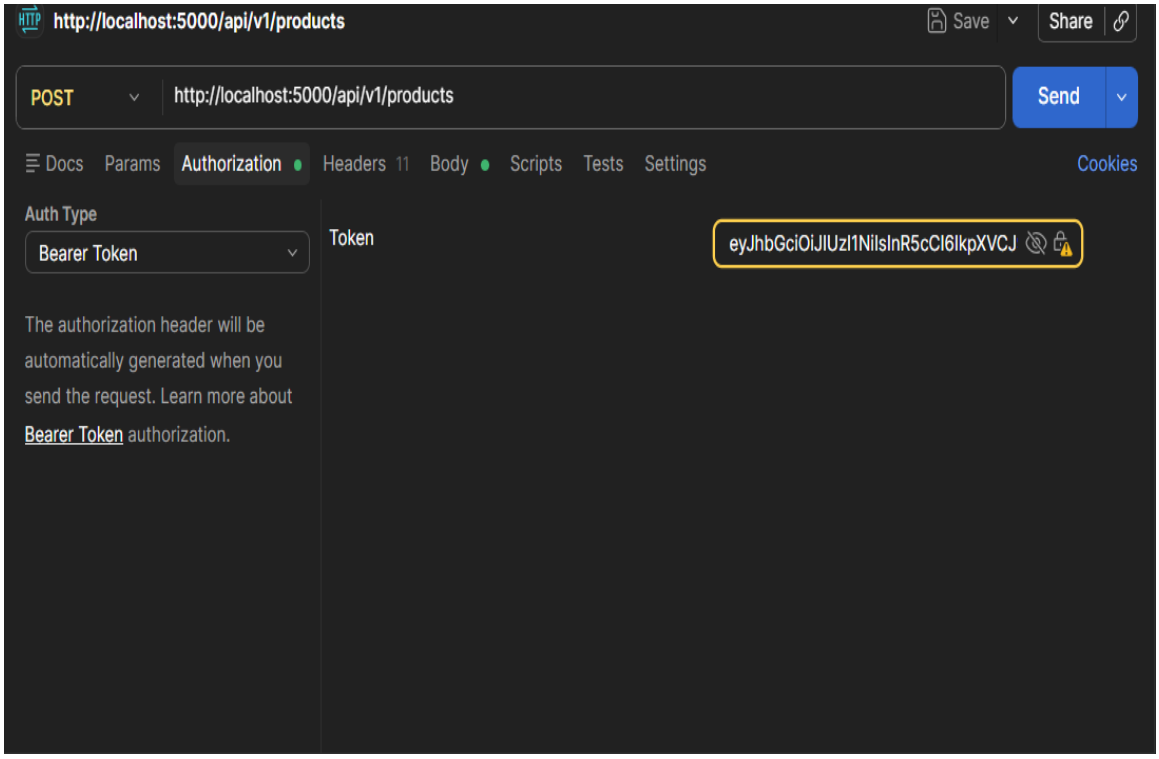
6. API Documentation

API	Method	Endpoint	Description
Register User	POST	/api/v1/auth/register	Create new user account
Login User	POST	/api/v1/auth/login	Authenticate user and return JWT token
Add Product	POST	/api/v1/products	Add new product
Get Products	GET	/api/v1/products	Retrieve all products
Update Product	PUT	/api/v1/products/{id}	Update product details
Delete Product	DELETE	/api/v1/products/{id}	Delete product

7. API Testing using Postman

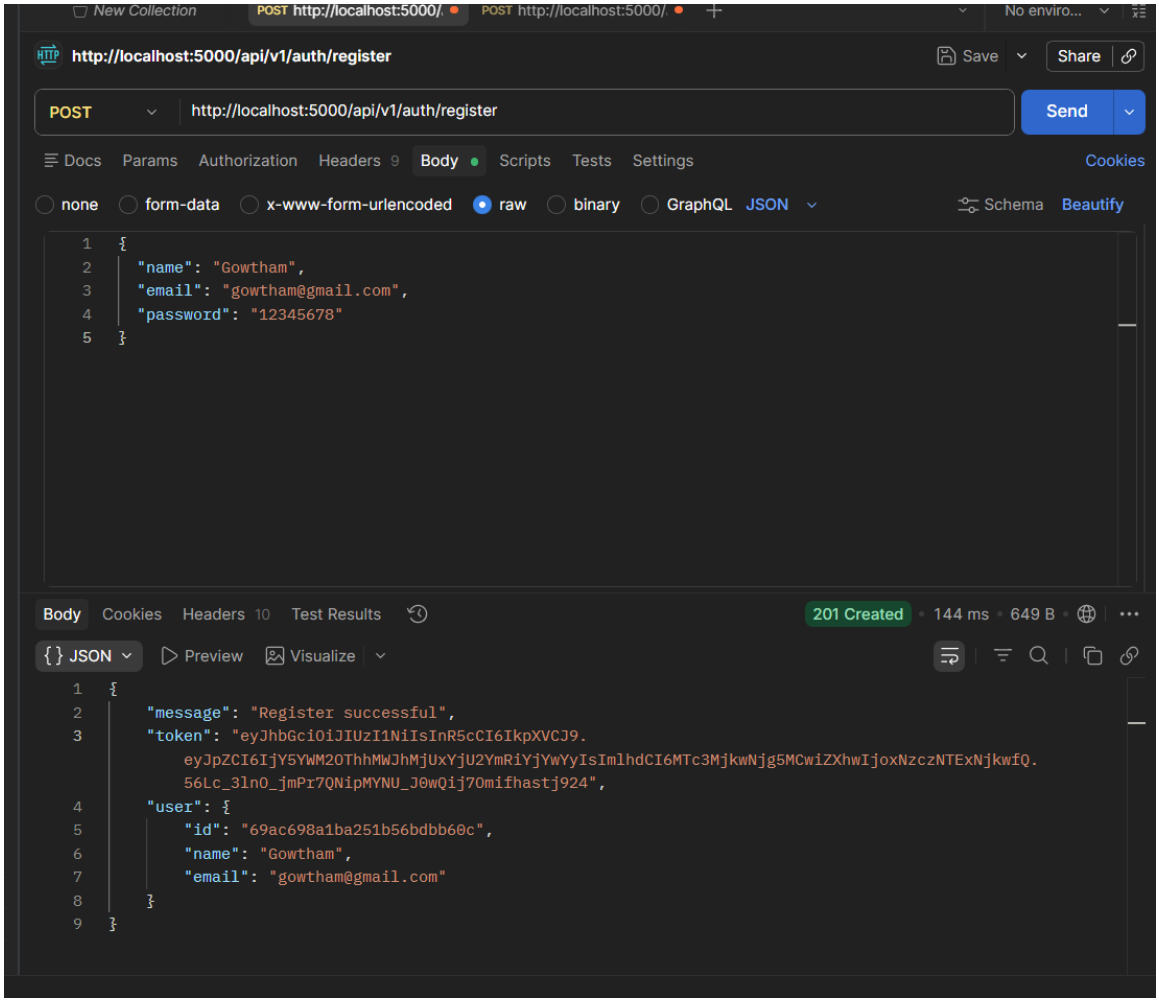
All APIs were tested using Postman. The login API generates a JWT token which must be used in the Authorization header for protected routes.

Authorization: Bearer <JWT_TOKEN>



8. Screenshots Section (Postman Testing)

Register API Response :



Login API Response :

New Collection | POST http://localhost:5000/ | POST http://localhost:5000/ | No enviro...

http://localhost:5000/api/v1/auth/login | Save | Share

POST | http://localhost:5000/api/v1/auth/login | Send

Docs | Params | Authorization | Headers 9 | Body | Scripts | Tests | Settings | Cookies

none | form-data | x-www-form-urlencoded | raw | binary | GraphQL | JSON | Schema | Beautify

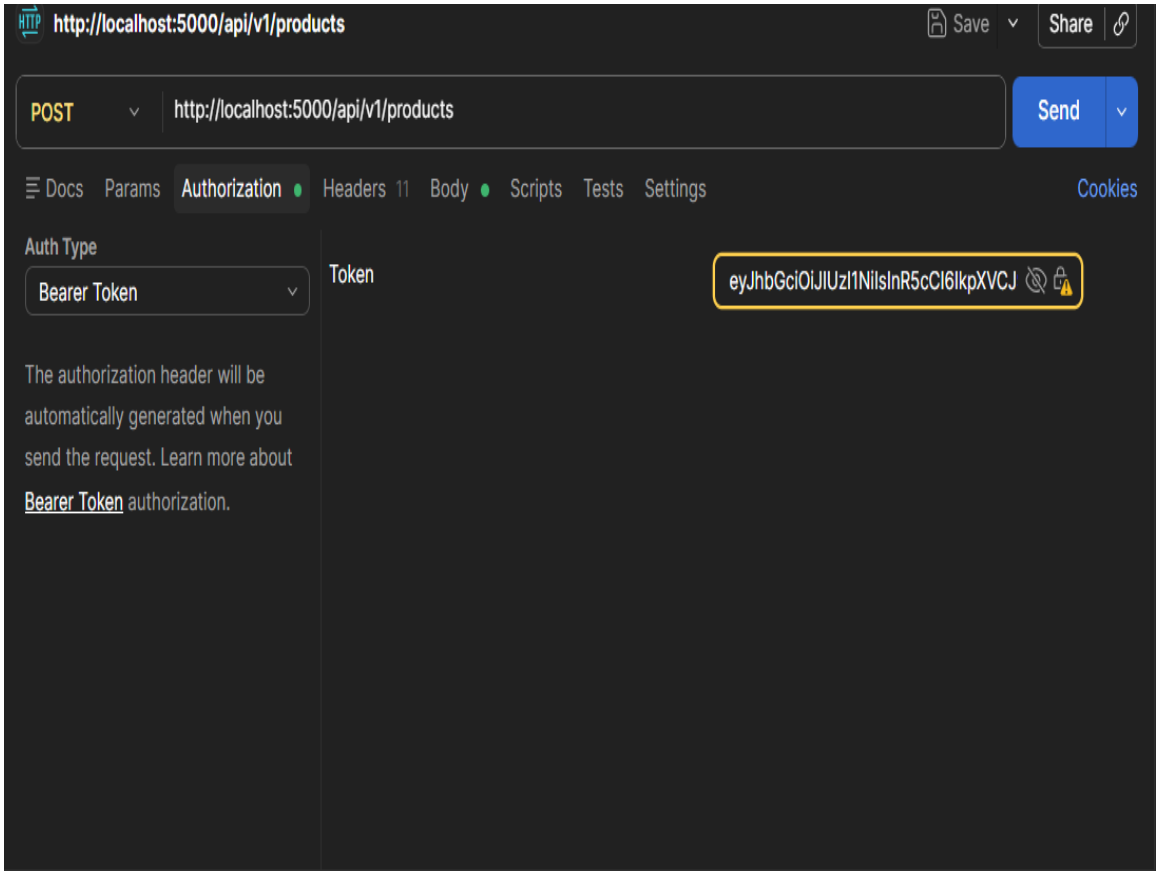

```
{
  "email": "gowtham@gmail.com",
  "password": "12345678"
}
```


Body | Cookies | Headers 10 | Test Results | 200 OK • 335 ms • 641 B

{} JSON | Preview | Visualize


```
{
    "message": "Login successful",
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
        eyJpZCI6IjY5YWM2OThhMWJhMjUxYjU2YmRiYjYwYyIsImlhdCI6MTc3MjkwNzAwNiwiZXhwIjoxNzczNTExODA2fQ.
        Vrtrb0bsB2yI5kWH24xqzUNV0vkVwcHQG10qCmqJw-A",
    "user": {
        "id": "69ac698a1ba251b56bdbb60c",
        "name": "Gowtham",
        "email": "gowtham@gmail.com"
    }
}
```

Add Product API :



Get Products API :

http://localhost:5000/api/v1/products

Save | Share

POST http://localhost:5000/api/v1/products Send

Docs | Params | Authorization | Headers 11 | Body | Scripts | Tests | Settings | Cookies

none | form-data | x-www-form-urlencoded | raw | binary | GraphQL | JSON | Schema | Beautify

```
{
  "name": "Laptop",
  "price": 65000,
  "category": "Electronics",
  "stock": 10
}
```

Body | Cookies | Headers 10 | Test Results | 201 Created | 14 ms | 604 B

JSON | Preview | Visualize

```
{
    "message": "Product created",
    "product": {
        "name": "Laptop",
        "price": 65000,
        "quantity": 0,
        "description": "",
        "userId": "69ac698a1ba251b56bdbb60c",
        "_id": "69ac6c7c1ba251b56bdbb610",
        "createdAt": "2026-03-07T18:20:44.059Z",
        "updatedAt": "2026-03-07T18:20:44.059Z",
        "__v": 0
    }
```

Update Product API :

http://localhost:5000/api/v1/products/69ac6c7c1ba251b56bdbb610

Save | Share

PUT http://localhost:5000/api/v1/products/69ac6c7c1ba251b56bdbb610 Send

Docs | Params | Authorization | Headers 10 | Body | Scripts | Tests | Settings | Cookies

none | form-data | x-www-form-urlencoded | raw | binary | GraphQL | JSON | Schema | Beautify

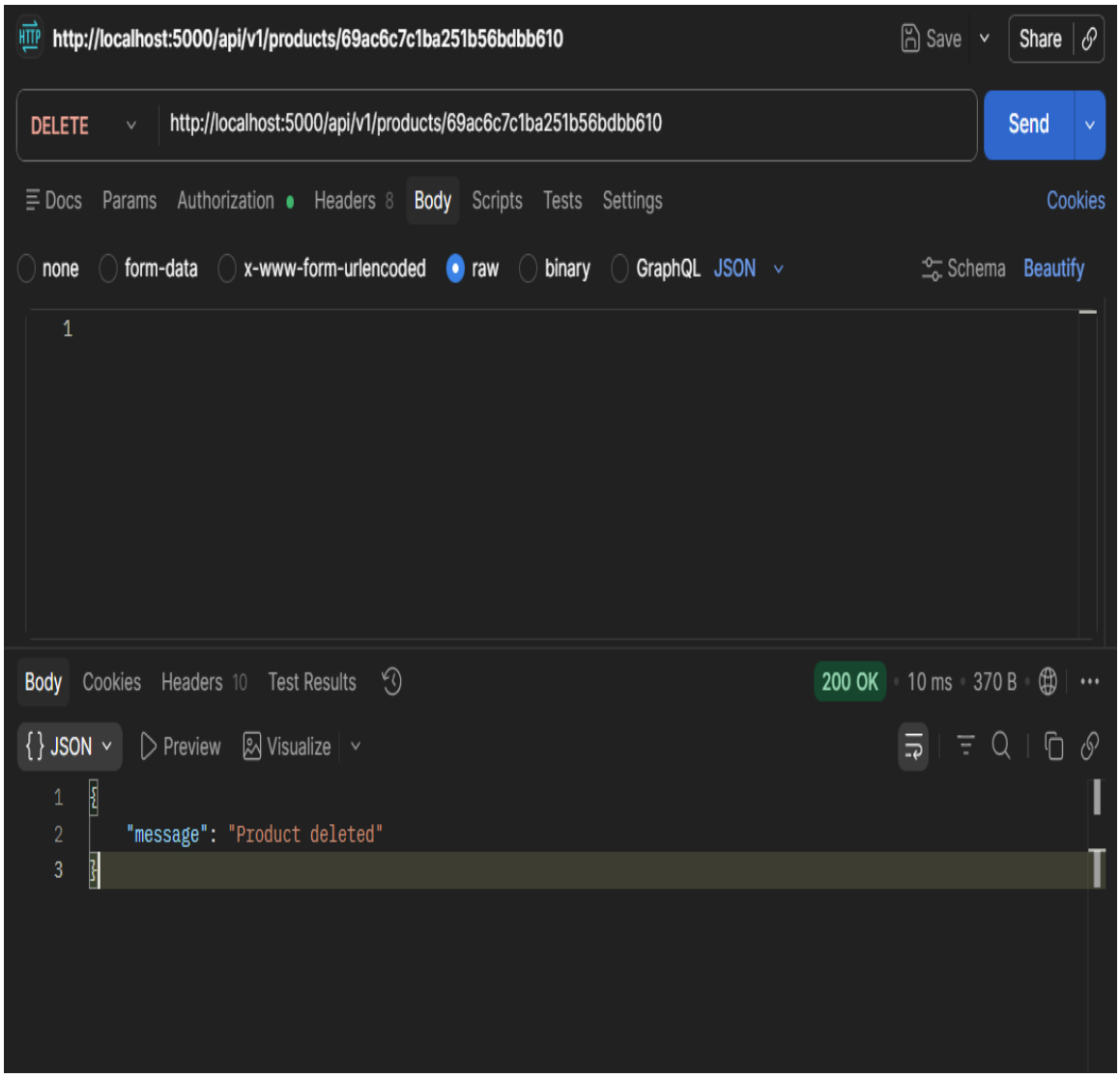
```
1 {
2   "name": "Gaming Laptop",
3   "price": 85000,
4   "stock": 5
5 }
```

Body | Cookies | Headers 10 | Test Results | 200 OK | 52 ms | 606 B

{} JSON | Preview | Visualize

```
1  {
2    "message": "Product updated",
3    "product": {
4      "_id": "69ac6c7c1ba251b56bdbb610",
5      "name": "Gaming Laptop",
6      "price": 85000,
7      "quantity": 0,
8      "description": "",
9      "userId": "69ac698a1ba251b56bdbb60c",
10     "createdAt": "2026-03-07T18:20:44.059Z",
11     "updatedAt": "2026-03-07T18:25:39.185Z",
12     "__v": 0
13   }
14 }
```

Delete Product API :



9. Conclusion

This project demonstrates the implementation of a secure backend API system using Node.js, Express.js and MongoDB. It provides complete product inventory management with authentication and CRUD functionality.